

Building a Reproducible Security Operations Centre Home Lab with an Open-Source SIEM

A Practical Study in Security Monitoring,
Endpoint Telemetry, and Alert Triage

Muhammad Ahmed Khalid

Personal Portfolio Project

This report documents a self-initiated technical project. It is not submitted for academic credit. It accompanies the project repository and is intended to demonstrate practical security-monitoring skills.

September 22, 2026

Abstract

Security Operations Centre (SOC) analysts depend on Security Information and Event Management (SIEM) platforms to detect and investigate threats, yet graduates entering the field rarely have hands-on experience with these tools. This project addresses that gap by designing, building, and documenting a fully functional SOC home lab using the open-source SIEM platform Wazuh.

The lab consists of a SIEM server and two monitored endpoints — a Windows host and a Linux host — deployed on an isolated virtual network. Enhanced endpoint telemetry is provided by Sysmon on Windows and the Linux Audit framework on Linux. The environment is used to generate security events, observe the resulting alerts, and demonstrate a realistic analyst triage workflow. A core aim of the project is reproducibility: every step is documented so a third party can rebuild the lab from the accompanying repository.

Keywords: SIEM, Security Operations Centre, Wazuh, threat detection, endpoint telemetry, log analysis, home lab.

Contents

Abstract	1
1 Introduction	4
1.1 Aim and Objectives	4
1.2 Research Questions	4
1.3 Contributions	4
1.4 Report Structure	5
2 Literature Review and Background Knowledge	5
2.1 Overview	5
2.2 Security Operations Centres	5
2.3 Security Information and Event Management	6
2.4 Log Management and Endpoint Telemetry	6
2.5 Open-Source SIEM Platforms	6
2.6 Threat Detection Approaches and the MITRE ATT&CK Framework	7
2.7 Virtualised Security Labs and the Skills Gap	7
2.8 Related Works	8
3 Methodology	8
3.1 Overall Approach	8
3.2 Environment Design	8
3.3 Tool Selection	9
3.4 Endpoint Telemetry Strategy	9
3.5 Attack Simulation Design	10
3.6 Triage and Documentation Approach	10
4 Implementation	11
4.1 Overview	11
4.2 Network and Virtualisation	11
4.3 Wazuh SIEM Deployment	11
4.4 Windows Endpoint	12
4.5 Linux Endpoint	12
4.6 Ingestion Verification and Baselining	13
5 Evaluation	13
5.1 Overview	13
5.2 Simulation 1 — Windows Local Account Creation and Privilege Escalation	13
5.3 Simulation 2 — Linux Local Account Creation with Sudo Privilege	14
5.4 Simulation 3 — Repeated Failed Authentication on Linux Endpoint	14
5.5 Simulation 4 — Negative Test, File Integrity Monitoring on Windows	15
5.6 Summary of Simulations	15
5.7 Evaluation Against Success Criteria	15
5.8 Evaluation Against Project Objectives	16
6 Discussion	16
6.1 Overview	16
6.2 RQ1: Can a Credible SOC Environment Be Built with Only Free and Open-Source Tools?	16
6.3 RQ2: What Additional Configuration is Required Beyond Defaults?	17
6.4 RQ3: To What Extent Does This Lab Represent a Professional SOC?	17

6.5	Challenges Encountered	18
6.6	Honest Reflection on Portfolio Value	18
7	Conclusions and Future Work	18
7.1	Summary	18
7.2	Answers to the Research Questions	19
7.3	Contribution	19
7.4	Future Work	19
7.5	Closing Note	20
	References	21

1 Introduction

Modern organisations generate enormous volumes of security-relevant data: authentication events, process executions, network connections, file changes, and system errors. Making sense of this data — and acting on it quickly enough to matter — is the work of a Security Operations Centre (SOC). At the centre of every SOC sits a Security Information and Event Management (SIEM) platform, which collects logs from across an estate, correlates them, and raises alerts an analyst can investigate.

For someone entering the cybersecurity profession, this creates a practical difficulty. Almost every junior SOC and security-analyst role expects familiarity with a SIEM, yet undergraduate study tends to teach the theory of detection and incident response without sustained hands-on exposure to the tools themselves. A graduate can describe what a SIEM does without ever having operated one.

This project was undertaken to close that gap directly. It designs, builds, and documents a working SOC home lab using only free and open-source software, producing both a functional environment and a reproducible record of how it was created.

1.1 Aim and Objectives

The **aim** of this project is to design, deploy, and operate a functional SOC home lab built on an open-source SIEM, capable of collecting and analysing security telemetry from multiple endpoints and supporting a realistic analyst triage workflow.

The aim is met through the following **objectives**:

- O1.** Deploy an open-source SIEM server on an isolated virtual network.
- O2.** Configure two monitored endpoints — a Windows host and a Linux host — each reporting to the SIEM.
- O3.** Enable enhanced host telemetry beyond default operating-system logs.
- O4.** Verify centralised log collection and establish a baseline of normal activity.
- O5.** Simulate a set of security events and observe the resulting alerts.
- O6.** Triage the alerts, classifying them and documenting the reasoning.
- O7.** Produce a fully reproducible record of the build so the lab can be rebuilt by a third party.

1.2 Research Questions

The project is framed around three research questions, revisited in the Discussion and Conclusions:

- **RQ1.** Can a credible SOC monitoring environment, capable of collecting endpoint telemetry and generating meaningful alerts, be built using only free and open-source tools?
- **RQ2.** What additional configuration is required beyond a default SIEM and operating-system installation to achieve detection-grade visibility?
- **RQ3.** To what extent does a single-machine virtualised lab realistically represent the work of a professional SOC, and where are the limits of that representation?

1.3 Contributions

This project contributes:

- A complete, working SOC home-lab build using Wazuh, deployed on an isolated network with Windows and Linux endpoints.
- A fully documented, step-by-step record enabling independent reproduction of the environment.
- A set of attack simulations paired with the alerts they produce and an analyst-style triage of each.
- An honest assessment of how closely such a lab represents real SOC work, including its limitations.

1.4 Report Structure

The remainder of this report is organised as follows. **Section 2** reviews the background literature on SOC, SIEM technology, logging, and security home labs, and positions this work against related projects. **Section 3** sets out the general methodology and design decisions. **Section 4** documents the specific implementation of the lab. **Section 5** evaluates whether the lab meets its objectives. **Section 6** discusses challenges and revisits the research questions, and **Section 7** concludes and identifies future work.

2 Literature Review and Background Knowledge

2.1 Overview

This section establishes the background needed to understand the project. It begins with the function of a Security Operations Centre and the role of the analyst, then examines the technology that underpins it: Security Information and Event Management (SIEM) platforms, log management, and endpoint telemetry. It compares open-source and commercial SIEM options, outlines the main approaches to threat detection and the MITRE ATT&CK framework, and considers the use of virtualised home labs for developing practical skills. The section closes with a review of related work and a statement of how this project differs from it.

2.2 Security Operations Centres

A Security Operations Centre (SOC) is the organisational function responsible for continuously monitoring an environment, detecting malicious or anomalous activity, and coordinating the response to security incidents. Rather than a single technology, a SOC is best understood as the combination of people, process, and tooling working together to maintain an organisation's security posture.

SOC analysts are commonly organised into tiers. Tier 1 analysts perform the initial triage of incoming alerts, deciding quickly whether an alert represents genuine malicious activity (a true positive) or harmless behaviour that merely resembles it (a false positive). Alerts that cannot be dismissed are escalated to Tier 2 analysts for deeper investigation, and the most serious or complex cases reach Tier 3 staff who carry out incident response and proactive threat hunting. This triage-first model means that the largest share of day-to-day SOC work consists of rapidly reviewing, classifying, and documenting alerts.

The structured handling of incidents that a SOC supports is described in NIST's incident-handling guidance, which frames incident response as a lifecycle of preparation; detection and analysis; containment, eradication and recovery; and post-incident activity [2]. Detection and analysis — the stage at which raw data becomes an actionable alert — is the stage this project is most concerned with, and it depends almost entirely on the SOC's central tooling: the SIEM.

2.3 Security Information and Event Management

A SIEM platform is the technological core of a SOC. The term combines two earlier categories: Security Information Management (SIM), concerned with the long-term collection, storage, and analysis of log data, and Security Event Management (SEM), concerned with the real-time monitoring and correlation of events. A modern SIEM unifies both.

In practice, a SIEM performs several core functions. It *collects* log and event data from many different sources across an estate. It *normalises* that data, parsing records written in many different formats into a consistent structure so they can be compared and searched. It *correlates* events, applying rules that recognise patterns spanning multiple log sources — for example, linking a series of failed logins followed by a success into a single suspicious sequence. It *alerts* analysts when activity matches a rule of concern, and it provides *dashboards and search* so that analysts can investigate. Finally, it *retains* data for the periods required by forensic investigation and regulatory compliance.

SIEM platforms have evolved considerably. Early systems were primarily log aggregators; later generations added correlation and alerting; current platforms increasingly incorporate user and entity behaviour analytics, threat-intelligence enrichment, and automated response. Throughout this evolution, the underlying premise has remained constant: an organisation cannot detect what it does not collect and analyse, and the SIEM is the instrument through which that collection and analysis happens.

2.4 Log Management and Endpoint Telemetry

Because detection ultimately rests on log data, the quality of that data is decisive. NIST's guidance on computer security log management emphasises that logs are essential not only for detecting attacks but also for understanding them after the fact, and that effective log management requires deliberate planning of what to collect and how to handle it [3]. National guidance makes a similar point: logging must be designed with security outcomes in mind rather than left to operating-system defaults [6].

This distinction — between default logging and security-grade logging — is central to the present project. On Windows, the standard Event Log captures useful records such as logon events and service changes, but it does not by itself provide the fine-grained visibility that behavioural detection requires. **Sysmon** (System Monitor), a free tool from Microsoft's Sysinternals suite, addresses this by logging detailed information about process creation, network connections, image loads, file-creation timestamps, and related activity, written to a dedicated event channel [5]. This additional telemetry is what allows a SIEM to reason about *behaviour* rather than merely about discrete system messages.

Linux provides an equivalent capability through the Linux Audit framework and its userspace daemon, **auditd**. By defining audit rules, an administrator can record security-relevant system activity — notably the execution of commands through the **execve** system call — at the kernel level. As with Sysmon on Windows, this elevates the endpoint from producing general operational logs to producing detection-grade telemetry.

The recurring principle across both platforms is that endpoint visibility is something that must be deliberately engineered. A SIEM connected only to default logs will see far less than one fed by purpose-configured endpoint telemetry.

2.5 Open-Source SIEM Platforms

SIEM capability is available both as commercial products and as open-source software. Commercial platforms such as Splunk, Microsoft Sentinel, and IBM QRadar are powerful and widely

deployed, but they are typically licensed by data volume, which can make them costly — and largely inaccessible for independent learning and experimentation.

Several capable open-source alternatives exist, including the Elastic Stack (often deployed as Elastic Security), Security Onion, Graylog, and Wazuh. This project uses **Wazuh**. Wazuh originated as a fork of **OSSEC**, a long-established open-source host-based intrusion detection system [7], and has since grown into a full SIEM and extended detection and response (XDR) platform [12]. A Wazuh deployment consists of a central *manager* that analyses incoming data and applies detection rules, an *indexer* that stores and searches events, a *dashboard* that presents them, and lightweight *agents* installed on monitored endpoints. Beyond log analysis and alerting, Wazuh provides file-integrity monitoring, configuration assessment, vulnerability detection, and the mapping of alerts to the MITRE ATT&CK framework.

For the purpose of building practical skills, an open-source platform offers clear advantages. It can be deployed at no cost, it can be examined and reconfigured freely, and — importantly — the concepts it teaches, such as agent-based collection, log parsing, rule-based correlation, and alert triage, transfer directly to commercial platforms. The specific product differs; the underlying discipline does not.

2.6 Threat Detection Approaches and the MITRE ATT&CK Framework

SIEM platforms detect threats using two broad approaches. **Signature- or rule-based** detection matches activity against known patterns of malicious behaviour; it is precise and explainable, but it can only catch what has been defined in advance. **Anomaly-based** detection flags deviations from an established baseline of normal behaviour; it can in principle catch novel activity, but it is prone to raising alerts on benign deviations. NIST’s guidance on intrusion detection and prevention systems describes both approaches and notes that practical deployments generally combine them, since each compensates for the other’s weakness [9].

A persistent operational problem for SOCs is *alert fatigue*: when detection produces a high volume of low-value or false-positive alerts, analysts become desensitised, and genuine threats risk being overlooked among the noise. This makes the *tuning* of detection — raising true positives while suppressing false positives — as important as detection itself.

To make detection systematic, the security community has increasingly adopted the **MITRE ATT&CK** framework: a structured, continually updated knowledge base of adversary tactics and techniques drawn from real-world observations of attacks [10]. ATT&CK gives defenders a common vocabulary and a way to measure detection *coverage* — mapping which adversary techniques an environment can and cannot detect. While the detailed application of ATT&CK belongs to the planned follow-on project on detection engineering, it is introduced here because it is the framework against which modern detection capability is now routinely assessed.

2.7 Virtualised Security Labs and the Skills Gap

The cybersecurity profession is widely recognised to face a shortage of skilled practitioners, and industry reporting consistently identifies a mismatch between the volume of security-relevant activity organisations must handle and the workforce available to handle it [11]. For new entrants specifically, the gap is often less about theoretical knowledge than about practical, tool-level experience.

Virtualised home labs are a well-established response to this problem. Using a hypervisor such as VirtualBox or VMware, a learner can build a self-contained environment of multiple virtual machines on a single physical computer, at no cost and in complete isolation from any production network. Such a lab can be broken, rebuilt, and experimented on freely. For defensive security in particular, a home lab allows a learner to stand up a SIEM, generate security events safely,

and practise the monitoring and triage workflow that a SOC role demands — experience that is otherwise difficult to obtain before employment. Frameworks such as the CIS Critical Security Controls reinforce the value of this hands-on focus by expressing good security practice in terms of concrete, implementable actions rather than abstract principles [1].

2.8 Related Works

Building security home labs is an established practice, and several reference projects exist. The most prominent is **DetectionLab** by Chris Long [4], an automated project that provisions a pre-built Windows domain instrumented with logging and monitoring tooling. DetectionLab is comprehensive and highly automated; its strength — one-command provisioning — is also a limitation for a learner, as the automation abstracts away the very build steps from which much of the understanding comes. A large body of community material, including vendor tutorials and blog write-ups, similarly documents Wazuh and Elastic-based lab builds, though such material varies in depth and is rarely presented as a structured, reproducible whole.

This project is positioned deliberately against that backdrop, and differs in three respects. First, **the build is performed and documented manually and in full**: reproducibility is treated as a primary deliverable, with each step recorded so that the understanding — not only the result — is transferable. Second, **the evaluation is explicitly honest about realism**: rather than presenting a home lab as equivalent to a production SOC, the report sets out clearly where a single-machine virtualised environment does and does not represent real operations. Third, **the project is designed as the first stage of a two-part progression**: it establishes the monitoring foundation on which a planned second project, focused on detection engineering with MITRE ATT&CK and Atomic Red Team [8], will build. The contribution is therefore not a novel tool, but a clear, honest, and reproducible account of how a credible SOC monitoring environment is constructed and operated using open-source software.

3 Methodology

3.1 Overall Approach

The project followed an iterative **build** → **verify** → **document** cycle. Each component of the lab was deployed in isolation, its correct operation was confirmed with a small controlled test before the next component was added, and the steps used were recorded at the time they were performed rather than reconstructed after the fact. This approach was chosen for three reasons. First, the fastest way to discover a broken assumption in a security build is to test the smallest possible piece before extending it. Second, documentation written contemporaneously is meaningfully more accurate than documentation written after the fact, particularly for the exact commands and configuration values that determine whether a deployment will reproduce. Third, the reproducibility of the lab was itself a project deliverable, so the documentation was treated as a first-class artefact throughout, not as a retrospective write-up.

3.2 Environment Design

The lab was designed around three constraints and one non-negotiable safety requirement. The constraints were: a single physical host with 16 GB of RAM, one hypervisor (Oracle VirtualBox), and the requirement that all components should be reproducible on any consumer-grade Windows machine. The safety requirement was that the environment must be fully isolated from any production network: no traffic from the lab could reach the wider Internet or the host network without deliberate configuration.

Isolation was achieved by placing all virtual machines on a VirtualBox *NAT network* named

SOC-Lab on the address range 10.0.10.0/24. This is distinct from VirtualBox’s default NAT mode: a NAT network permits VM-to-VM communication on the same segment while still translating outbound traffic through the host, which was necessary for initial package downloads but was subsequently locked down. The address plan allocated one address per role: 10.0.10.3 for the SIEM manager, 10.0.10.4 for the Windows endpoint, and 10.0.10.5 for the Linux endpoint. Keeping the address assignments deterministic simplified both the Wazuh agent enrolment and the later analysis, since the source of every log line could be identified from its IP alone.

The 16 GB host RAM constraint meant that not all VMs could run simultaneously under load. In practice, the Wazuh manager was always running (its role required continuous operation to receive telemetry), and the two endpoints were run in alternation for heavy testing.

3.3 Tool Selection

Every tool used in the project was selected against three criteria: it had to be free or open-source, it had to be genuinely used in industry, and its core concepts had to transfer to commercial equivalents rather than being specific to a hobbyist stack. The chosen stack is summarised in Table 1.

Category	Chosen Tool	Rationale
Hypervisor	Oracle VirtualBox	Free, cross-platform, widely used in student and lab contexts.
SIEM / XDR	Wazuh 4.14.5	Open-source, deployed in production by real organisations, forked from OSSEC with an active development community.
Windows telemetry	Sysinternals Sysmon with SwiftOnSecurity config	Industry standard for detection-grade Windows logging; the SwiftOnSecurity baseline is the de facto sensible default.
Linux telemetry	Linux Audit (auditd) with <code>execve</code> rules	The Linux equivalent of Sysmon: kernel-level command execution visibility.
Endpoint OS (Windows)	Microsoft WinDev 2407 evaluation VM	Legitimate free-to-use Windows 11 image intended for developer testing.
Endpoint OS (Linux)	Ubuntu 26.04 LTS Desktop	Common, well-documented server distribution with LTS support.
Documentation	Markdown + LaTeX	Markdown for build docs (renderable natively on GitHub), LaTeX for the formal report.

Table 1: Tool selection with rationale.

The most significant of these choices is **Wazuh**, which was preferred over Splunk Free (data cap and no long-term retention), Elastic Security (more complex initial deployment for a single learner), and Security Onion (more focused on network detection than endpoint monitoring in the shape required for this project). Wazuh’s out-of-the-box combination of manager, indexer, dashboard, and agent, with a pre-populated ruleset that already recognises common attack patterns, offered the best balance of depth and setup effort for a single-analyst home lab.

3.4 Endpoint Telemetry Strategy

A recurring theme in the literature review was that default operating-system logs are insufficient for meaningful behavioural detection. The methodology therefore treated *deliberate endpoint telemetry enhancement* as a mandatory step, not an optional one. Both endpoints were provisioned with their platform’s equivalent of detection-grade logging.

On **Windows**, this meant installing Sysmon with the SwiftOnSecurity baseline configuration and then extending the Wazuh agent configuration to forward the `Microsoft-Windows-Sysmon/Operational` event channel to the manager. Without this addition, the Wazuh agent would have received only the standard Windows Security Event Log; with it, the agent received the process-creation, network-connection, and file-modification records that modern detection depends on.

On **Linux**, the equivalent step was installing the `auditd` daemon and adding audit rules for the `execve` system call, tagged with the `audit-wazuh-c` key so command-execution events could be distinguished from other audit traffic. Wazuh's default Linux agent configuration already parses `/var/log/audit/audit.log` once `auditd` is present, so no further Wazuh-side configuration was required.

This paired approach — Sysmon on Windows, `auditd` on Linux — reflects real SOC practice, where both platforms need their own deliberate telemetry enhancement to produce comparable visibility.

3.5 Attack Simulation Design

Attack simulations were designed against three principles.

First, safety. All simulations used benign emulated behaviour rather than live malware or real exploitation. Where a technique required the creation of an artefact (a user, a group, a file entry), the artefact was reversible; where it required a command, the command was benign in isolation and only meaningful in aggregate. This is consistent with community guidance on running attack simulations in home labs and with the intent of tooling such as Atomic Red Team [8].

Second, coverage across the two platforms. The simulations were distributed across Windows and Linux endpoints, and across multiple ATT&CK tactic categories (specifically Persistence, Privilege Escalation, Credential Access, and Defence Evasion), rather than concentrated on a single technique family. This ensured that the resulting evaluation could speak to lab coverage as a whole, not merely to how well one technique was detected.

Third, at least one intentionally negative test. One simulation was designed with the expectation that the default Wazuh configuration would *not* detect the activity, because a lab that only ever produces successful detections is not being honestly stress-tested. A negative result, when it identifies a specific configuration gap that can be closed, is as valuable as a positive result. This is discussed further in the Evaluation (Section 5) and Discussion (Section 6).

3.6 Triage and Documentation Approach

Each simulation was followed by a structured triage exercise mimicking the work of a Tier 1 SOC analyst. The steps applied to every simulation were:

- T1.** Observe the Wazuh dashboard within a defined window (typically one minute) after the simulated activity was performed.
- T2.** Identify the alerts produced, capturing rule ID, rule description, and severity level for each.
- T3.** Classify each alert as a true positive (correctly identified malicious-shaped activity), a false positive (a benign event erroneously flagged), or expected background noise.
- T4.** Map the observed activity to the corresponding MITRE ATT&CK technique(s) where applicable.
- T5.** Capture a screenshot of the dashboard state as evidence.
- T6.** Write a full analyst-style incident report for the simulation, treating the events as if they were unplanned.

Producing an incident report for every simulation — rather than only recording the raw alerts — was a deliberate methodological choice. It forces a level of structured reasoning about impact, response, and remediation that is not required when merely observing that an alert appeared. The incident report artefacts are stored in the project’s `incident-reports/` directory and are considered part of the deliverable set.

4 Implementation

4.1 Overview

This section walks through the actual build of the lab: the network, the SIEM manager, the two endpoints, the endpoint telemetry enhancements, and the verification that logs flow end to end. It corresponds to the full step-by-step documentation stored in the project repository under `docs/`, distilled here into narrative form. The precise commands, configuration snippets, and screenshots referenced in this section are held in those source documents so this report can remain readable while still being independently reproducible.

4.2 Network and Virtualisation

Oracle VirtualBox 7.2.8 was used to host all virtual machines. A dedicated *NAT network* named `SOC-Lab` was created on the address range `10.0.10.0/24` as described in Section 3.2. The NAT network mode was chosen deliberately over the more permissive Bridged mode: VMs on a NAT network can reach each other but cannot be reached from, or directly reach, the host network without deliberate port forwarding. This provides the isolation guarantee that a security lab must offer by default.

Three virtual machines were provisioned onto this network:

- The Wazuh server VM at `10.0.10.3`.
- The Windows endpoint VM at `10.0.10.4`.
- The Linux endpoint VM at `10.0.10.5`.

Fixed addressing was preferred over DHCP for a lab of this size, so that sources in the Wazuh dashboard would always resolve to the same host without needing to cross-reference DHCP leases.

4.3 Wazuh SIEM Deployment

The Wazuh server was deployed from the official `wazuh-4.14.5.ova` appliance provided by Wazuh Inc. The appliance is a pre-built all-in-one deployment containing the three Wazuh components required to operate the platform:

- **wazuh-manager** — the analysis and rule engine that receives agent telemetry, applies detection rules, and generates alerts.
- **wazuh-indexer** — the storage and search layer, built on OpenSearch, that persists alerts and events so they can be queried and displayed.
- **wazuh-dashboard** — the browser-based user interface, built on OpenSearch Dashboards, that presents alerts and enables analyst workflows.

Using the appliance rather than performing a from-source install was a pragmatic choice: it reduced the initial deployment to VM import and network attachment, which allowed the project to move on to the more instructive work of endpoint configuration and detection engineering. The trade-off is that the appliance is opaque compared to a from-source deployment; a production

build would follow Wazuh's documented distributed installation procedure so component sizing could be tuned per role.

After boot, the appliance's default credentials were rotated, its network interface was confirmed on 10.0.10.3, and the dashboard was reached at <https://10.0.10.3> from the host machine. The dashboard's first-run wizard was used to generate agent deployment commands for the two endpoints.

4.4 Windows Endpoint

The Windows endpoint was provisioned from Microsoft's `WinDev2407Eval` appliance (Windows 11), a free evaluation image intended for developer testing. This choice avoided the licensing and installation overhead of a manual Windows install while remaining fully representative of a modern Windows endpoint.

Three configuration steps were performed after the VM was joined to the `SOC-Lab` network:

First, the **Wazuh agent** was installed using the PowerShell one-liner generated by the manager's Deploy new agent wizard. The wizard tailored the command with the manager IP (10.0.10.3) and the assigned agent name (`Windows-Endpoint`). The agent service was started with `NET START WazuhSvc` and appeared as **Active** in the manager's agent list within seconds.

Second, **Sysmon** was installed with the `SwiftOnSecurity` baseline configuration. Sysmon writes to a dedicated Windows event channel, `Microsoft-Windows-Sysmon/Operational`, which is separate from the Windows Security event log and requires explicit inclusion in the Wazuh agent's configuration to be forwarded. The following block was added to the agent's `ossec.conf` inside the `<ossec_config>` element:

```
<localfile>
<location>Microsoft-Windows-Sysmon/Operational</location>
<log_format>eventchannel</log_format>
</localfile>
```

The agent service was restarted, and within one minute the manager began receiving events under the `sysmon`, `sysmon_event1`, and `sysmon_event_3` rule groups. This confirmed that the `SwiftOnSecurity` configuration was producing telemetry and that the Wazuh manager's Sysmon decoder was correctly parsing the incoming events.

Third, a **baseline of normal activity** was observed for approximately twenty-four hours. This served two purposes: it validated that the agent would continue reporting stably across VM restarts, and it established a reference of what "clean" telemetry from the endpoint looked like, so that subsequent simulation-generated alerts could be distinguished from routine noise.

Full step-by-step commands, screenshots, and verification outputs are recorded in `docs/03-windows-endpoint.m` in the project repository.

4.5 Linux Endpoint

The Linux endpoint was provisioned by performing a clean Ubuntu 26.04 LTS Desktop install from the official ISO. Unlike Wazuh and Windows, both of which were imported from vendor-provided appliances, the Linux install was carried out manually. This was a deliberate choice: the Linux install exposed first-hand the number of small decisions (partitioning, timezone, initial user creation) that shape a system's behaviour and that an appliance import hides.

The Wazuh agent was installed on the Ubuntu VM using the manager's wizard-generated `wget` and `dpkg` command, then enabled and started as a systemd unit. It appeared as **Active** in the dashboard with `agent.id 002`.

The critical configuration step on Linux was adding **auditd command-execution monitoring**. Wazuh's default Linux agent configuration reads system logs from `/var/log/syslog`, `/var/log/auth.log`, and, if present, `/var/log/audit/audit.log`. However, Ubuntu Desktop does not include the Linux Audit framework by default, so **auditd** was installed via **apt**, and audit rules were added to track every invocation of the **execve** system call — which is called whenever any command is executed on the system:

```
-a exit,always -F arch=b64 -S execve -k audit-wazuh-c
-a exit,always -F arch=b32 -S execve -k audit-wazuh-c
```

The **-k audit-wazuh-c** key identifies these events as command-execution audits, so they can be selected in Wazuh queries. After the rules were loaded with **augenrules -load**, the Wazuh agent was restarted and began forwarding **audit.log** entries alongside the default log sources.

At this point, both endpoints were producing detection-grade telemetry, and the milestone identified in the project overview as M4 (both endpoints delivering enhanced telemetry to the SIEM) was met. Full commands, output, and verification screenshots are recorded in `docs/04-linux-endpoint.md`.

4.6 Ingestion Verification and Baselining

Once both endpoints were reporting, three verification checks were performed before any attack simulation was attempted. First, the number of events per minute in the dashboard's *Threat Hunting* view was observed for each endpoint over a five-minute window and cross-referenced against known activity on that endpoint (for example, opening a text editor on the Linux endpoint produced a visible spike in **execve** events). Second, the alert-level distribution was checked: a healthy Wazuh deployment shows a long tail of Level 1 to Level 3 informational events, a small number of Level 5 to Level 8 alerts, and rare Level 12+ high-severity events, which was observed. Third, a manual query for **rule.description: "Sysmon"** on the Windows agent, and **rule.description: "execve"** on the Linux agent, was used to confirm that both endpoints' enhanced telemetry was not merely being ingested but was actually reachable through the dashboard's query interface.

With these verifications passed, the lab was considered operational and ready for the attack simulations described in Section 5.

5 Evaluation

5.1 Overview

This section evaluates the completed lab against the objectives set out in Section 1.1 and the success criteria in the Initial Project Overview. Four attack simulations were executed across the two endpoints, and each was subject to the structured triage process described in Section 3.6. For every simulation, the executed activity, the alerts produced, the analyst classification, and the mapped MITRE ATT&CK technique(s) are documented. Full-length analyst-style incident reports for each simulation are held in the project's **incident-reports/** directory as **INC-2026-001** through **INC-2026-004**. Screenshots referenced below are stored in the **screenshots/** directory and form the evidence base for the evaluation.

5.2 Simulation 1 — Windows Local Account Creation and Privilege Escalation

Activity performed. From an elevated PowerShell session on the Windows endpoint, a new local account **testuser** was created and immediately added to the local Administrators group:

```
net user testuser P@sswOrd123! /add
net localgroup administrators testuser /add
```

ATT&CK mapping. T1136.001 (Create Account: Local Account) and T1078.003 (Valid Accounts: Local Accounts).

Detection result. Within sixty seconds, the Wazuh dashboard produced a cluster of correlated alerts. The most operationally significant were *Administrative Privilege Assigned* (Level 12), *Users Group Changed* (Level 8), and *Discovery activity spawned by PowerShell execution* (Sysmon-based, Level 5). The full alert cluster, including several lower-level supporting events, is documented in INC-2026-001.

Classification. True positive across all alerts. The correlation across Sysmon process telemetry (net.exe spawned by powershell.exe) and Windows Security event log entries (4720 for account creation, 4732 for privileged group addition) provided independent confirmation of the same underlying activity from two different data sources.

5.3 Simulation 2 — Linux Local Account Creation with Sudo Privilege

Activity performed. From a terminal session on the Linux endpoint, a new local user was created, given a password, and added to the sudo group:

```
sudo useradd -m testuser
echo "testuser:P@sswOrd123!" | sudo chpasswd
sudo usermod -aG sudo testuser
```

ATT&CK mapping. T1136.001 (Create Account: Local Account), T1548.003 (Abuse Elevation Control Mechanism: Sudo and Sudo Caching), and T1078.003 (Valid Accounts: Local Accounts).

Detection result. Within sixty seconds, alerts fired for *New user added to the system* (Rule 5902, Level 8), *New group added to the system* (Rule 5901, Level 8), and *Successful sudo to ROOT executed* (Rule 5402, Level 3), along with the corresponding PAM session events. Full details are recorded in INC-2026-002.

Classification. True positive across all alerts. The detection pattern is the direct Linux counterpart to Simulation 1, and the analyst triage flow was near-identical, which suggests a single unified runbook covering both platforms would be a natural detection engineering follow-up.

5.4 Simulation 3 — Repeated Failed Authentication on Linux Endpoint

Activity performed. Repeated incorrect passwords were entered at a sudo prompt for a valid local account, mimicking the pattern of a password-guessing attempt.

ATT&CK mapping. T1110.001 (Brute Force: Password Guessing).

Detection result. Each failed attempt produced a paired alert under Rule 5557 (*unix_chkpwd: Password check failed*, Level 5) and Rule 5503 (*PAM: User login failed*, Level 5). No higher-severity aggregate alert was raised by the default ruleset, even though the individual failures accumulated visibly on the dashboard. Full details are recorded in INC-2026-003.

Classification. True positive individually; however, the absence of a correlation rule that promotes a cluster of failed attempts into a higher-severity aggregate alert is a documented detection engineering gap. In a production environment, a Tier 1 analyst would rely on a dashboard spike or a custom aggregate rule rather than paging on each individual Level 5 event.

5.5 Simulation 4 — Negative Test, File Integrity Monitoring on Windows

Activity performed. A line was appended to the Windows hosts file using PowerShell:

```
Add-Content -Path "C:\Windows\System32\drivers\etc\hosts" -Value "127.0.0.1 test-fim.local"
```

ATT&CK mapping. T1565.001 (Stored Data Manipulation), with adjacent relevance to T1556 (Modify Authentication Process) and to hosts file abuse used in DNS-redirection attacks.

Detection result. No File Integrity Monitoring alert was produced within the observation window. Other unrelated telemetry (service startup type changes, executable creation events) continued to appear on the dashboard, confirming the agent was still forwarding events normally.

Classification. *Negative result*, documented as a detection gap in INC-2026-004. The root cause is the default Wazuh syscheck configuration on Windows, which runs periodic scans (default interval: 12 hours) rather than real-time file-change monitoring. Real-time monitoring for a specific path requires adding `realtime="yes"` to the corresponding `<directories>` entry in the agent configuration. The incident report proposes the specific configuration change that would close this gap, together with a wider recommendation to enable real-time FIM selectively on a defined set of security-critical files across both platforms.

5.6 Summary of Simulations

The four simulations are summarised in Table 2. The combination of true positives (simulations 1, 2, 3) and the deliberate negative result (simulation 4) demonstrates both the strengths and the tunable gaps of a default-configured Wazuh deployment.

Sim	Activity	Endpoint	ATT&CK	Result
1	Local account creation and privilege escalation	Windows	T1136.001, T1078.003	True positive; correlated alert cluster within 60s
2	Local account creation with sudo privilege	Linux	T1136.001, T1548.003, T1078.003	True positive; correlated alert cluster within 60s
3	Repeated failed sudo authentication	Linux	T1110.001	True positive per event; no default aggregate correlation
4	Hosts file modification, FIM test	Windows	T1565.001	Negative result; documented gap, remediation proposed

Table 2: Summary of attack simulations executed in the lab.

5.7 Evaluation Against Success Criteria

The Initial Project Overview defined success in four specific terms. Each is addressed below.

Criterion 1: The SIEM is operational with two or more endpoints actively reporting. *Met.* The Wazuh manager, indexer, and dashboard were deployed and remained operational throughout the evaluation period, with two enrolled endpoints (Windows and Linux) both appearing as **Active** in the dashboard and both forwarding both default and enhanced telemetry.

Criterion 2: At least four distinct security events have been simulated and detected. *Met.* Four distinct simulations were executed as described in this section, three producing true-positive alerts and the fourth producing a documented negative result (which is itself a valid detection engineering finding, not a failure).

Criterion 3: Each detection is documented with triage notes and a true/false-positive classification. *Met.* Every simulation has a corresponding full analyst-style incident report (INC-2026-001 through INC-2026-004) with executive summary, timeline, detection detail, technical analysis, impact assessment, response actions, and recommendations. Each alert has been classified.

Criterion 4: A third party can reproduce the lab by following the published guide. *Met, subject to the usual caveats of hypervisor and OS version drift.* The build documentation (docs/01 through docs/04) is written as a step-by-step reproducible guide, records exact versions of the software used, and is published to the project's public GitHub repository alongside this report and the incident reports.

5.8 Evaluation Against Project Objectives

The seven objectives from Section 1.1 were met as follows: O1 (deploy an open-source SIEM), O2 (configure two monitored endpoints), and O3 (enable enhanced host telemetry) are demonstrated in Section 4. O4 (verify centralised log collection and baseline normal activity) was completed prior to running any simulation, as described in Section 4.6. O5 (simulate security events) and O6 (triage the alerts and classify them) are covered by the four simulations documented above. O7 (produce a fully reproducible record of the build) is met by the combination of this report, the docs/ build documents, and the accompanying GitHub repository.

6 Discussion

6.1 Overview

This section revisits the three research questions posed in Section 1.2, reflects on the challenges encountered during the build, and discusses the limitations of the lab in the context of a professional SOC environment. The intent is to be honest about what the project does and does not represent, so its value as a portfolio artefact rests on defensible claims rather than on overstated equivalence.

6.2 RQ1: Can a Credible SOC Environment Be Built with Only Free and Open-Source Tools?

The evidence from the build supports a qualified yes. Wazuh, Sysmon, and auditd, deployed on VirtualBox with two commodity endpoint operating systems, produced a working SIEM environment capable of ingesting detection-grade telemetry, applying a mature rule set out of the box, and raising meaningful alerts within seconds of simulated attacker activity. Every tool used is deployed in production by real organisations, and the concepts practised in the lab — agent-based collection, log parsing, rule-based correlation, alert triage — transfer directly to commercial platforms.

The qualification is that "credible" here means credible for the purposes of individual learning and portfolio demonstration, not for the purposes of running a live security programme for a real organisation. Multi-tenant scalability, high availability, long-term compliance retention, integration with commercial threat intelligence feeds, and 24-hour analyst rotation are all absent. What the lab does credibly represent is the daily operational core of a SOC analyst's work: interpreting alerts, correlating events across sources, classifying activity, and producing structured triage documentation.

6.3 RQ2: What Additional Configuration is Required Beyond Defaults?

This turned out to be the most instructive question of the three. A default installation of Wazuh, a default installation of Windows, and a default installation of Ubuntu, connected together, produced far less useful telemetry than a naive reading of the vendor documentation would suggest. Three specific configuration additions were required to reach detection-grade visibility:

1. On Windows, Sysmon had to be installed and its dedicated event channel (`Microsoft-Windows-Sysmon/Operational`) had to be explicitly forwarded by the Wazuh agent through an added `<localfile>` block in `ossec.conf`. Without this, the agent received only standard Windows Security event log entries, which do not include the process, network, and file-creation records that modern behavioural detection depends on.
2. On Linux, `auditd` was not present in the default Ubuntu Desktop install. It had to be installed, and audit rules for the `execve` system call had to be added and loaded before the Wazuh agent could see command-execution events.
3. Simulation 4 identified a third gap: the default Wazuh syscheck File Integrity Monitoring configuration on Windows runs on a twelve-hour scan cycle rather than in real-time. A modification to a security-sensitive file such as the hosts file will eventually be detected, but not with the latency a live SOC would need. Closing this gap requires adding `realtime="yes"` to the relevant `<directories>` entries.

The recurring theme is that vendor defaults optimise for broad coverage with low overhead, not for security-critical corner cases. A deployment that relies only on defaults will have predictable blind spots, and the work of detection engineering is largely the work of identifying and deliberately closing those blind spots. This directly motivates the planned Project 2 on detection engineering with MITRE ATT&CK.

6.4 RQ3: To What Extent Does This Lab Represent a Professional SOC?

Realistically, the lab represents perhaps twenty per cent of a professional SOC's operational surface, and does so along one axis: the individual analyst's interaction with a SIEM. The remaining eighty per cent, honestly enumerated, includes:

- **Scale.** A production SIEM ingests events at rates several orders of magnitude beyond what two lab endpoints produce. Alert fatigue, triage prioritisation, and tuning at scale are among the hardest and most important skills in a real SOC, and the lab cannot exercise them.
- **Team workflow.** There is no Tier 2 or Tier 3 escalation, no shift handover, no on-call rotation, no incident coordination across multiple analysts. The lab is a single-analyst environment.
- **Network detection.** All monitoring in the lab is host-based. A production SOC also relies on network-tier detection through IDS/IPS platforms such as Suricata or Zeek and on full packet capture. This is out of scope here and identified as future work in Section 7.
- **Cloud and SaaS telemetry.** Modern SOC's increasingly work with cloud audit logs, SaaS activity, and identity provider events. The lab is entirely on-premises virtualised infrastructure.
- **Threat intelligence enrichment.** Alerts in the lab are not enriched with external context such as IP reputation, malware family attribution, or campaign intelligence. In a production environment this enrichment is a major driver of triage decisions.
- **Response tooling.** There is no EDR quarantine, no SOAR playbook automation, and no case management ticketing. Response actions in the lab were performed by hand.

Within its scope, however, the lab does honestly reflect one specific piece of SOC work: what an analyst does in the first sixty seconds after an alert appears. That is a small share of a SOC's

total surface but a significant share of a Tier 1 analyst's daily work, and it is the piece that is hardest to practise without a working SIEM to practise on.

6.5 Challenges Encountered

Three challenges shaped the build in ways worth documenting for future readers.

Resource constraint on the host machine. The 16 GB of RAM on the host was tight for running three virtual machines concurrently under any sustained load. In practice the two endpoints were alternated for heavy testing while the Wazuh manager remained continuously running. This is a legitimate constraint for a home lab of this class, but it would prevent any experiment that required both endpoints active simultaneously (for example, lateral movement simulations between them).

Vulnerability-detector disk consumption. During extended idle periods between build sessions, the Wazuh manager's vulnerability-detector cache expanded until it consumed the entire 25 GB manager disk. The symptom was manager service failure with a critical configuration error that traced back to `touch: No space left on device`. The fix was to clear the `/var/ossec/queue/vd_updater/` and `/var/ossec/queue/vd/` directories, restart the indexer and manager, and (in the longer term) either grow the manager disk or disable the vulnerability-detector module in `ossec.conf`. This incident is worth recording because it is the kind of operational maintenance a lab owner learns only by encountering, and because it illustrates how a SIEM component can fail silently between active use.

Configuration file structure across versions. Some steps in public Wazuh guides refer to configuration paths or agent commands that have changed between major versions. Working from the vendor's own documentation for the specific version installed (4.14.5) rather than from third-party tutorials generally avoided this issue, and this experience reinforces a lesson from the methodology: recording exact software versions at build time is not optional.

6.6 Honest Reflection on Portfolio Value

The value of this project as a portfolio artefact is not that it makes the author qualified to operate a production SOC on its own; it does not. The value is that it provides evidence of three specific things which are frequently claimed by graduate applicants but rarely demonstrated:

- **Hands-on SIEM operation**, rather than only textbook familiarity, evidenced by the deployed environment and the documented triage of real alerts.
- **Awareness of the gap between defaults and detection-grade configuration**, evidenced by the deliberate endpoint telemetry work and by the negative-result FIM simulation.
- **Structured technical documentation and analyst-style reporting**, evidenced by the incident reports and by this report itself, both of which are held to a standard that is suitable for handover to a colleague.

These are the specific competencies most difficult for a graduate applicant to demonstrate from coursework alone, and they are the specific competencies this project was designed to develop.

7 Conclusions and Future Work

7.1 Summary

This project designed, built, documented, and evaluated a Security Operations Centre home lab using entirely free and open-source tooling. The lab consists of a Wazuh SIEM manager and two monitored endpoints — one Windows and one Linux — on an isolated virtualised network.

Both endpoints run deliberate detection-grade telemetry: Sysmon with the SwiftOnSecurity baseline on Windows, and the Linux Audit framework with `execve` monitoring on Linux. Four attack simulations covering persistence, privilege escalation, credential access, and detection configuration were executed and triaged against the built environment, each producing either a documented true-positive alert cluster or a deliberately documented negative result. Full analyst-style incident reports were written for every simulation, and the entire build was recorded as a reproducible step-by-step guide alongside a formal report (this document).

The seven project objectives set out in Section 1.1 were all met, and the four success criteria stated in the Initial Project Overview were all satisfied.

7.2 Answers to the Research Questions

RQ1. A credible SOC monitoring environment can be built with only free and open-source tools, provided that "credible" is scoped to individual learning and portfolio demonstration rather than to running a production security programme. The Wazuh stack in particular is genuinely representative of a live SIEM in both operation and concept.

RQ2. Meaningful detection requires deliberate configuration beyond defaults on both endpoint platforms: Sysmon plus a Wazuh-side `<localfile>` block for Windows, and `auditd` with `execve` rules for Linux. In addition, real-time File Integrity Monitoring on security-critical paths requires an explicit `realtime="yes"` directive, without which FIM operates on a twelve-hour scan cycle. Vendor defaults optimise for coverage with low overhead, not for security-critical corner cases, and closing those corner cases is a large share of what detection engineering is.

RQ3. A single-machine virtualised lab represents perhaps twenty per cent of a professional SOC's operational surface, and does so along the axis most relevant to the Tier 1 analyst role: the first sixty seconds after an alert appears. Scale, team workflow, network detection, cloud telemetry, threat intelligence enrichment, and response automation are all absent, and this is documented openly in the Discussion rather than glossed over.

7.3 Contribution

The specific contribution of this project is a fully documented, reproducible SOC home lab build using open-source tools, paired with a set of triaged attack simulations that include both positive and deliberately negative results, and with analyst-style incident reports for each simulation. The project is deliberately positioned as the first stage of a two-part progression: it establishes the monitoring foundation on which a planned second project, focused on detection engineering with MITRE ATT&CK and Atomic Red Team, will build.

7.4 Future Work

Several extensions were identified during the project and are noted here for future development. They fall into three groups.

Detection engineering (Project 2). The planned follow-on project will focus on custom rule development, mapping detection coverage against MITRE ATT&CK, and using Atomic Red Team to execute a broader and more systematic set of attack techniques. The two specific detection gaps identified in this project — the absence of an aggregate rule for repeated failed authentication (Simulation 3) and the lack of real-time FIM on the Windows hosts file (Simulation 4) — are both natural starting points for that work.

Additional telemetry sources. The current lab is host-based only. Adding network-tier detection through Suricata or Zeek integrated with Wazuh would extend coverage to lateral movement and command-and-control traffic that host telemetry alone cannot see. Enabling

the Wazuh vulnerability-detector on a schedule (with the disk consumption issue discussed in Section 6 mitigated) would add vulnerability visibility to the existing detection surface. Integrating a threat intelligence feed for IOC enrichment would move the lab closer to the enrichment workflows real SOCs depend on.

Response automation and case management. All response actions in the current lab were performed by hand. Adding Wazuh’s active-response module, or integrating a lightweight SOAR platform such as Shuffle for playbook automation, would demonstrate the response side of the SOC lifecycle in the same practical way that this project demonstrates the detection side. Adding a lightweight case management or ticketing layer would let the analyst workflow be exercised end to end.

7.5 Closing Note

The core aim of this project was to close, for the author personally, the gap between theoretical knowledge of SIEM operations and practical hands-on experience of it. That gap has been closed, and the resulting build and documentation are offered as both a self-contained portfolio artefact and as a foundation for further work in detection engineering. The environment stands ready for that follow-on work; the disciplines it required to build — deliberate telemetry design, structured triage, honest documentation, and the willingness to run and record negative tests — are the same disciplines the follow-on work will demand at greater depth.

References

- [1] Center for Internet Security. Cis critical security controls, version 8. <https://www.cisecurity.org/controls>, 2021. Accessed: 2026.
- [2] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide (sp 800-61 revision 2). Nist special publication, National Institute of Standards and Technology (NIST), 2012.
- [3] Karen Kent and Murugiah Souppaya. Guide to computer security log management (sp 800-92). Nist special publication, National Institute of Standards and Technology (NIST), 2006.
- [4] Chris Long. Detectionlab. <https://github.com/clong/DetectionLab>, 2023. Accessed: 2026.
- [5] Microsoft Corporation. Sysmon — windows sysinternals. <https://learn.microsoft.com/sysinternals/downloads/sysmon>, 2025. Accessed: 2026.
- [6] National Cyber Security Centre (NCSC). Introduction to logging for security purposes. <https://www.ncsc.gov.uk/guidance/introduction-logging-security-purposes>, 2025. Accessed: 2026.
- [7] OSSEC Project. Ossec — open source hids. <https://www.ossec.net>, 2025. Accessed: 2026.
- [8] Red Canary. Atomic red team. <https://github.com/redcanaryco/atomic-red-team>, 2025. Accessed: 2026.
- [9] Karen Scarfone and Peter Mell. Guide to intrusion detection and prevention systems (idps) (sp 800-94). Nist special publication, National Institute of Standards and Technology (NIST), 2007.
- [10] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. Mitre att&ck: Design and philosophy. Technical report, The MITRE Corporation, 2020.
- [11] Verizon. Data breach investigations report (dbir). <https://www.verizon.com/business/resources/reports/dbir/>, 2024. Accessed: 2026.
- [12] Wazuh Inc. Wazuh documentation. <https://documentation.wazuh.com>, 2025. Accessed: 2026.